

Parcial plantilla

```
1  public class ReportGenerator {
2      private String type;
3      public ReportGenerator(String type) { this.type = type; }
4
5      public void generateReport(Document document) {
6          if ("PDF".equals(type)) {
7              // crear documento y configurar metadatos
8              DocumentFile docFile = new DocumentFile();
9              docFile.setTitle(document.getTitle());
10             String authors = "";
11             for (String author : document.getAuthors()) {
12                 authors += ", " + author;
13             }
14             docFile.setAuthor(authors);
15             docFile.setContentType("application/pdf");
16             docFile.setPageSize("A4");
17
18             // crear exportador y setear el contenido
19             PDFExporter exporter = new PDFExporter();
20             byte[] content = exporter.generatePDFFile(document);
21             docFile.setContent(content);
22
23             // guardar el documento
24             this.saveExportedFile(docFile);
25         } else if ("XLS".equals(type)) {
26             // crear documento y configurar metadatos
27             DocumentFile docFile = new DocumentFile();
28             docFile.setTitle(document.getTitle());
29             String authors = "";
30             for (String author : document.getAuthors()) {
31                 authors += ", " + author;
32             }
33             docFile.setAuthor(authors);
34             docFile.setContentType("application/vnd.ms-excel");
35             docFile.setSheetName(document.getSubtitle());
36
37             // crear exportador y setear el contenido
38             ExcelWriter writer = new ExcelWriter();
39             byte[] content = writer.generateExcelFile(document);
40             docFile.setContent(content);
41
42             // guardar el documento
```

```

43         this.saveExportedFile(docFile);
44     }
45 }
46 }

1 class ReportGeneratorTest {
2     ReportGenerator generatorPDF;
3     ReportGenerator generatorXLS;
4     Document document;
5
6     @BeforeEach
7     void setUp() {
8         document = new Document("Informe");
9         document.addAuthor("Carlos");
10        document.addAuthor("Ana");
11
12        generatorPDF = new ReportGenerator("PDF");
13        generatorXLS = new ReportGenerator("XLS");
14    }
15
16    @Test
17    void testPDF() {
18        generatorPDF.generateReport(document);
19
20        // aca se detallan los asserts
21    }
22
23    @Test
24    void testXLS() {
25        generatorXLS.generateReport(document);
26
27        // aca se detallan los asserts
28    }
29 }

```

Code smells

- **Comments:** Se necesitan comentarios para explicar que hace cada metodo
 - Lineas 7, 18, 23, 26, 37 y 42
- **Switch statements:** Hay dos condicionales preguntando por el tipo, lo cual es poco escalable
 - Lineas 6 y 25
- **Long method:** Es un metodo que tiene un monton de responsabilidades juntas

- **Duplicate code:** Tanto en el if como en el else

- Línea 7 - 14 y 26 - 33 idénticos
- Línea 21 - 24 y 40-43 idénticas

1. Empezamos aplicando *Extract Method* sobre las líneas 7 - 14 y 21 - 24 para generalizarlo ya que se repite en las líneas 26 - 33 y 40 - 43. Esto nos eliminaría una parte de código repetido y generaría nombres autoexplicativos para poder eliminar la necesidad de comentarios. También aplicamos *Consolidate Duplicate Conditional fragments* para sacarlo fuera del if y el else

```
1  public class ReportGenerator {
2      private String type;
3      public ReportGenerator(String type) { this.type = type; }
4
5      public void generateReport(Document document) {
6          DocumentFile docFile = crearYConfigurarDocumento();
7          crearExportadorYSetearContenido(type, docFile)
8          this.saveExportedFile(docFile);
9      }
10
11     private DocumentFile crearYConfigurarDocumento(Document document){
12         DocumentFile docFile = new DocumentFile();
13         docFile.setTitle(document.getTitle());
14         String authors = "";
15         for (String author : document.getAuthors()) {
16             authors += ", " + author;
17         }
18         docFile.setAuthor(authors);
19         return docFile;
20     }
21
22     private void crearExportadorYSetearContenido(String type, Document
docFile){
23         if ("PDF".equals(type)) {
24             docFile.setContentType("application/pdf");
25             docFile.setPageSize("A4");
26             PDFExporter exporter = new PDFExporter();
27             byte[] content = exporter.generatePDFFile(document);
28             docFile.setContent(content)
29         } else if ("XLS".equals(type)) {
30             docFile.setContentType("application/vnd.ms-excel");
31             docFile.setSheetName(document.getSubtitle());
32             ExcelWriter writer = new ExcelWriter();
33             byte[] content = writer.generateExcelFile(document);
```

```

34         docFile.setContent(content)
35     }
36 }
37 }

```

Ahora aplicamos *Replace Conditional with Polymorfism*. Esto es lo mas adecuado ya que una vez instanciado no se puede cambiar el tipo del ReportGenerator. El primer paso es hacer un *Extract method* del condicional

```

1  public class ReportGenerator {
2      private String type;
3      public ReportGenerator(String type) { this.type = type; }
4
5      public void generateReport(Document document) {
6          DocumentFile docFile = crearYConfigurarDocumento();
7          crearExportadorYSetearContenido(type, docFile)
8          this.saveExportedFile(docFile);
9      }
10
11     private DocumentFile crearYConfigurarDocumento(Document document){
12         DocumentFile docFile = new DocumentFile();
13         docFile.setTitle(document.getTitle());
14         String authors = "";
15         for (String author : document.getAuthors()) {
16             authors += ", " + author;
17         }
18         docFile.setAuthor(authors);
19         retrun docFile;
20     }
21
22     private void crearExportadorYSetearContenido(String type, Document
docFile){
23         if ("PDF".equals(type)) {
24             docFile.setContentType("application/pdf");
25             docFile.setPageSize("A4");
26             PDFExporter exporter = new PDFExporter();
27             byte[] content = exporter.generatePDFFile(document);
28             docFile.setContent(content)
29         } else if ("XLS".equals(type)) {
30             docFile.setContentType("application/vnd.ms-excel");
31             docFile.setSheetName(document.getSubtitle());
32             ExcelWriter writer = new ExcelWriter();
33             byte[] content = writer.generateExcelFile(document);
34             docFile.setContent(content)
35         }

```

```
36     }
37 }
```

Ahora seguimos la mecanica, por cada rama del condicional creamos una subclase, movemos el metodo de la rama del condicional hacia la subclase, compilamos y testeamos

```
1  public class ReportGenerator {
2      private String type;
3      public ReportGenerator(String type) { this.type = type; }
4
5      public void generateReport(Document document) {
6          DocumentFile docFile = new DocumentFile();
7          crearYConfigurarDocumento(document);
8          crearExportadorYSetearContenido(docFile)
9          this.saveExportedFile(docFile);
10     }
11
12     private DocumentFile crearYConfigurarDocumento(Document document){
13         docFile.setTitle(document.getTitle());
14         String authors = "";
15         for (String author : document.getAuthors()) {
16             authors += ", " + author;
17         }
18         docFile.setAuthor(authors);
19         return docFile;
20     }
21
22     public abstract void crearExportadorYSetearContenido(DocumentFile df);
23 }
24
25 public class PDFReportGenerator extends ReportGenerator{
26     @Override
27     public void crearExportadorYSetearContenido(){
28         docFile.setContentType("application/pdf");
29         docFile.setPageSize("A4");
30         PDFExporter exporter = new PDFExporter();
31         byte[] content = exporter.generatePDFFile(document);
32         docFile.setContent(content)
33     }
34 }
35
36 public class XLSReportGenerator extends ReportGenerator{
37     @Override
38     public void crearExportadorYSetearContenido(){
39         docFile.setContentType("application/vnd.ms-excel");
```

```
40         docFile.setSheetName(document.getSubtitle());
41         ExcelWriter writer = new ExcelWriter();
42         byte[] content = writer.generateExcelFile(document);
43         docFile.setContent(content)
44     }
45 }
```

Hacemos una subclase y un metodo por cada una de las ramas del condicional y hacemos el metodo arriba abstracto

Aca tambien aplicamos remove unused parameter con el type, ya que el polimorfismo nos soluciona no preguntar mas por el tipo